

게임 플레이어 위치 예측을 위한 CNN 모델 연구 보고서

문제 상황 및 데이터 구성

온라인 팀 게임 환경에서 **10명의 플레이어 상태와 시간 정보를 담은 3차원 시계열 데이터**(shape: 10명 players × 다차원 status × 시간)가 주어졌습니다. 이 데이터를 기반으로 **특정 시점(예: 60초)에 플레이어 7번의 위치**를 예측하는 딥러닝 모델을 개발하고자 합니다. 예측 대상인 위치 좌표는 정규화된 2D 좌표 ((x, y), 범위 0~1)이며, 예측 정확도 평가에는 좌표 간 **L2 거리 기반 손실 (Mean Squared Error, MSE)**를 고려합니다.

데이터 구조를 세부적으로 보면:

- **1축 (player):** 플레이어 인덱스 0~9 (총 10명).
- **2축 (status):** 각 플레이어의 상태 피쳐들 (예: 위치 좌표, 체력, 스킬 쿨타임, 이동 속도 등 다차원 특징 벡터).
- **3축 (time):** 시간 흐름에 따른 상태 값. 기본적으로 1초 간격의 0초부터 60초까지 timeline이지만, **특정 이벤트 발생 시 비정기적인 추가 타임스탬프**가 기록되어 시계열 간격이 불규칙해질 수 있습니다.

이러한 데이터와 과제를 해결하기 위해 고려해야 할 중요한 이슈들은 다음과 같습니다:

1. **불규칙한 타임스탬프와 결측값 처리:** 이벤트로 인해 촬영된 추가 시점 데이터가 존재할 때, 해당 시계열의 공백 (null)을 어떻게 처리할 것인가 (예: 선형 보간 vs. 그대로 남기기 등).
2. **플레이어의 고유 캐릭터 ID 활용:** 각 플레이어가 조종하는 캐릭터의 종류(ID)를 모델 입력에 통합할 필요가 있을 때, **어떻게 표현할 것인지** (원-핫 인코딩, 임베딩 등) 및 CNN에서 이를 효과적으로 반영하는 방법.
3. **3차원 텐서의 CNN 입력 구조:** `player × status × time` 형태를 CNN에 넣을 때, **player와 status 차원을 통합하여 다루어도 될지** 혹은 **특정 축 재구성 또는 전처리가 필요한지**.

다음 섹션들에서는 위 이슈들에 대한 **유사 연구 사례와 해결 방법**을 조사하고, 효과적인 **모델 아키텍처 설계, 데이터 전처리 전략, 학습 방법 및 손실 함수** 설정에 대한 권고사항을 제시합니다.

유사 연구 사례 및 적용 사례

- **스포츠 플레이어 트래젝토리 예측:** 실제 스포츠 경기 데이터에서도 여러 선수들의 위치나 움직임을 예측하는 연구가 진행되어 왔습니다. 예를 들어 NFL 미식축구 경기에서 수비 선수의 미래 위치를 예측하는 모델에 CNN-LSTM 구조를 활용한 사례가 있습니다 ^①. 해당 연구에서는 각 입력 피쳐 시계열에 1차원 CNN을 적용해 **시간적 패턴을 추출**하고, 그 출력들을 LSTM에 연결하여 **미래 위치 시퀀스**를 예측했습니다. 이 모델은 **다수 선수의 동시 움직임을 예측**하는 데 성공적으로 적용되었으며, **1D-CNN으로 특징 추출 후 위치 좌표를 회귀**하는 구조를 사용했습니다 ^{② ③}. 이러한 접근은 우리 문제에도 시사점을 주며, 시계열 CNN이 **순차 데이터 특징 추출**에 유용함을 보여줍니다.

- **Temporal Convolutional Network (TCN):** 시계열 예측 영역에서 최근 각광받는 TCN 구조는 **딜레이드 컨볼루션(dilated CNN)**을 통해 긴 시간 범위의 의존성을 효과적으로 학습할 수 있음을 보여줍니다 ^{④ ⑤}. RNN 없이도 **병렬적인 1D 합성곱 연산**만으로도 장기 패턴을 포착하여 전력 수요나 주가 예측 등에 높은 성능을 낸 사례들이 보고되었습니다. TCN은 **순차 전체를 한 번에 처리**하므로 길이가 긴 시계열에서도 학습이 효율적이며, MSE 등의 손실을 시퀀스 단위로 계산하여 최적화합니다 ^⑥. 본 과제에서도 **60초 내의 변화 추세**를 잡아내는 데 TCN 개념을 활용할 수 있습니다 (예: 여러 층의 1D-CNN으로 receptive field를 확장하여 60초까지 커버).

- **e스포츠/게임 데이터 예측:** MOBA와 같은 게임에서 승리 팀 예측이나 특정 이벤트 발생 예측에도 다변량 시계열 모델이 사용된 바 있습니다. 다만 플레이어 위치 좌표 예측에 초점을 맞춘 공개 사례는 드물지만, 일반적으로 게임 로그 시계열에 RNN(LSTM)이나 CNN을 적용하는 연구들이 존재합니다⁷. 예컨대 **League of Legends** 데이터로 팀 승률을 예측하는 모델에서 플레이어별 골드, 경험치 등의 시계열을 병렬 입력 채널로 하여 1D-CNN으로 처리한 사례 등이 보고되었습니다. 이러한 경우에도 각 플레이어의 데이터 채널을 통해 합성곱이 동시에 여러 플레이어의 상태 변화를 학습할 수 있도록 합니다.

以上的 사례들은 시계열 CNN이 RNN 대안으로 쓰이거나 함께 활용되어 다중 요소가 얹힌 동적 상황을 예측하는 데 유용함을 보여줍니다. 특히, 1D-CNN을 이용한 시계열 특징 추출과 병렬 채널 활용 전략은 본 과제의 모델 설계에 직접 참고할 수 있습니다.

모델 아키텍처 구성 방안

이제 주어진 과제에 적합한 CNN 기반 모델 아키텍처를 제안합니다. 목표는 0~60초 사이의 다인 플레이어 상태 변화를 입력으로 60초 시점의 7번 플레이어 위치(x,y)를 회귀 예측하는 것입니다. 주요 구성 아이디어는 다음과 같습니다:

1. 입력 텐서 형태 결정:

먼저 3차원 텐서 입력을 CNN에 맞게 구조화해야 합니다. 두 가지 접근을 고려할 수 있습니다:

- **(A) Player × Status 축 통합 (채널화):** $10 \times \text{status} \times \text{time}$ 텐서를 (time, features) 형태로 변환하여 1D-CNN 입력으로 사용합니다. 예를 들어, 10명의 플레이어 * status차원 수만큼 feature를 가지는 하나의 다변량 시계열로 간주하는 것입니다⁸. 이 때 time축(예: 60 steps)을 따라 1D 합성곱 커널을 적용하고, feature들은 채널(channel)로 취급합니다. 이렇게 하면 합성곱 필터가 시간축 국소영역에서 모든 플레이어의 상태 채널을 동시에 참고하여 특징을 학습할 수 있습니다. Cross-Validated 답변에서도 “다변량 시계열의 추가 변수는 이미지의 채널처럼 다루는 것이 적절하다”고 권고하고 있습니다⁸. 실제로 CNN 필터는 여러 채널의 입력을 가중합하여 출력을 내므로, 각 필터가 여러 플레이어 간 상관관계를 학습할 수 있습니다.
- **(B) 2D-CNN (players × time)을 공간으로 간주:** 두 번째로, players를 세로축, time을 가로축으로 보고 행렬 형태 ($10 \times T$)의 시계열 이미지를 생성하는 방법입니다. status의 각 피처별로 이런 행렬을 하나의 채널로 취급하여 2D-CNN을 적용할 수 있습니다⁹. 예를 들어 각 상태 피처마다 (10×61) 이미지를 만들고, 이를 다채널(예: status 개수 채널) 이미지로 입력하면 2D 합성곱으로 시간과 플레이어 차원을 모두 고려한 필터링이 가능합니다. 논문에서도 “다변량 시계열은 2차원 이미지 데이터 구조와 같으므로 CNN 적용이 가능하다”는 언급이 있었지만⁹, 이 접근에는 주의가 필요합니다. 플레이어 간에는 순서나 거리 개념이 없기 때문에 2D 커널이 인접 플레이어들을 특별히 의미있는 조합으로 다루지 않을 수 있습니다⁸. 따라서 players 축을 공간으로 간주하는 2D-CNN은 필터가 임의의 인접 플레이어 패턴을 학습할 위험이 있습니다.

두 접근 중 일반적으로 권장되는 것은 (A)와 같이 players/status를 채널로 보고 1D-CNN을 사용하는 것입니다. 이는 각 플레이어의 피처들을 분리 채널로 투입하므로, 합성곱 필터가 시간축에만 국한되어 학습하고 플레이어별 특성 결합은 네트워크가 채널 가중치로 자연스럽게 학습하게 합니다⁸. 다만 (B) 접근도 팀 구분이나 공간적인 근접 관계 등을 반영하여 플레이어 순서를 재조정한다면 보조적으로 실험해볼 수 있습니다.

2. 합성곱 계층 구성:

1D-CNN 아키텍처로 가정하고 구체적인 구성을 제시하면 다음과 같습니다:

- 입력은 shape (T, F)로서 T=60 (또는 추가 이벤트 포함 시간 step), F는 $10 * \text{status_dim}$ (플레이어 10명의 모든 상태 피처 개수)입니다. 예를 들어 status_dim이 5라면 F=50인 시계열 특징 벡터입니다.
- **합성곱 층을 다층으로 쌓기:** 시간축 방향으로 합성곱을 수행하는 여러 층을 둡니다. 첫 Conv1D 층은 적당한 필터 크기 (예: kernel_size=3~5)로 짧은 시간 패턴을 포착하고 ReLU 등의 활성화 함수를 거칩니다¹⁰. 동일한

1D 합성곱을 깊게 3~5계층 누적하여 점진적으로 **더 긴 시간 패턴**을 학습할 수 있습니다. 매 층마다 충분한 필터 수 (예: 64 등)를 사용해 표현 능력을 확보합니다. (필요에 따라 BatchNorm이나 Dropout을 적용하여 과적합을 방지하고 수렴을 돕습니다 ¹¹.)

- **풀링/디일레이션**: 일반적인 시계열 분류에서는 MaxPooling으로 차원 축소를 하기도 하지만, **미래 위치 회귀**에서는 시퀀스의 **시간 순서 정보 손실을 최소화**하는 것이 중요합니다. 따라서 풀링은 지나치게 사용하지 않거나, TCN처럼 **디일레이션 기법**을 사용하여 **풀링 없이도 receptive field를 확장**하는 것이 좋습니다. 예컨대 Dilated Conv를 사용하면 풀링 없이도 몇 계층으로 60-step 이상의 시야를 확보할 수 있습니다. 풀링을 사용하면 다면 마지막 예측에서 **종합적인 추세를** 보게 하는 **Global Average Pooling** 정도가 적합할 것입니다 ¹².

- **멀티헤드 구조 (대안)**: 만약 각 플레이어의 상태 시계열 패턴이 매우 이질적이어서 개별 처리 후 통합이 필요하다고 판단되면, **멀티헤드 CNN** 구조도 고려할 수 있습니다. 예를 들어 10명의 플레이어 각각에 대해 별도의 1D-CNN 분기를 두고 (각 분기 입력 shape $(T, \text{status_dim})$), 마지막에 이 분기들의 출력을 concatenate하여 예측에 활용하는 것입니다 ¹³ ¹⁴. 이렇게 하면 **플레이어별로 최적화된 필터**를 학습시킬 수 있지만, 데이터가 풍부하지 않은 경우 분기별 파라미터가 늘어나 오히려 과적합 위험이 있습니다. 보통은 shared CNN으로 충분하므로 기본 구조는 단일 CNN 경로로 설정합니다.

- **완전연결 출력층**: 합성곱 계층들의 출력을 펼치거나(pooling 결과 등) Flatten하여 **Dense 레이어**에 연결합니다 ¹⁵. 회귀 문제이므로 마지막 Dense 노드는 2개 (예: 출력 (x, y))로 설정하고 별도 활성화함수 없이 선형 출력으로 두는 것이 일반적입니다. (출력좌표 범위가 0~1로 정규화되어 있으므로, 모델이 예측값을 자연스럽게 이 범위 내로 맞추길 기대할 수 있습니다. 필요하다면 **sigmoid** 활성화를 붙여 0~1 범위를 강제할 수도 있지만, MSE 손실 최적화 과정에서 자동으로 범위에 맞춰질 가능성이 큼니다.)

以上 구조는 시계열 길이가 비교적 짧은(60) 경우에 적합한 **전형적인 1D-CNN 회귀 모델**입니다. 이러한 CNN만으로도 짧은 시계열 예측에는 충분한 성능을 낼 수 있으며, 실제 연구에서도 **CNN 추출 특징 + 좌표 회귀**로 구성되어 정확도를 높인 사례가 있습니다 ¹. 만약 **더 긴 시퀀스나 복잡한 상호작용**까지 고려해야 한다면 LSTM이나 Transformer와의 결합도 고려해볼 수 있겠지만, 여기서는 단순화를 위해 CNN 기반으로도 충분한 모델링이 가능하도록 설계합니다.

3. 플레이어 ID 특성 통합:

모델 구조 상 중요한 부분은 **플레이어의 고유 캐릭터 ID(feature)**를 어떻게 입력에 녹여낼 것인가입니다. 각 플레이어는 게임 내에서 고유한 캐릭터(챔피언 등)를 사용하고 있고, 이 캐릭터의 능력이나 특성이 플레이어의 이동 패턴이나 전투 양상에 영향을 줄 수 있습니다. 따라서 “**플레이어 7이 누구인가**”에 대한 정보를 모델이 알도록 만드는 것이 필요합니다. 이를 구현하는 방법으로는 **원-핫 인코딩**이나 **임베딩 벡터** 두 가지가 대표적입니다:

- **원-핫 인코딩**: 캐릭터 종류 개수가 M이라고 할 때, 각 플레이어에게 크기 M의 one-hot 벡터를 할당합니다 (해당 캐릭터 index만 1, 나머지 0). 이 벡터를 그 플레이어의 상태 feature에 추가로 붙여주는 것입니다. 예를 들어 원래 status feature가 5차원이고 M=20종 캐릭터라면, 새로운 status feature는 5+20=25차원이 되고, 그 중 20차원이 ID를 나타내는 one-hot입니다. 이 방법은 구현이 간단하지만, **캐릭터 종류가 많으면 차원이 매우 커지고 희소해**지는 단점이 있습니다.

- **임베딩(Embedding) 기법**: 캐릭터 ID를 **밀집 벡터(dense vector)**로 표현합니다. 임베딩 층을 두어 ID(정수)를 입력받아 예컨대 4~8 차원의 continuous vector로 매핑하는 것입니다. 임베딩 차원은 실험적으로 정할 수 있으며, one-hot 대비 훨씬 소형의 표현이고, 학습을 통해 **유사 캐릭터는 임베딩 공간에서 가까워지도록** 조정될 수 있는 장점이 있습니다 ¹⁶. 허깅페이스 시계열 변환기 문서에서도 “**정적 범주형 특징은 임베딩으로 학습되어 시계열 값에 더해진다**”고 설명하고 있습니다 ¹⁶. 이는 본 과제의 “플레이어 ID”에 그대로 적용할 수 있는 개념입니다.

임베딩 활용 방식: 임베딩 벡터를 모델 입력에 통합하는 일반적인 방법은 **상태 feature에 concatenate**하는 것입니다. 구체적으로, **플레이어별로** 자신의 ID 임베딩을 시간축 전체에 걸쳐 붙여줍니다. 예를 들어 플레이어 7의 임베딩 벡터가 $(e_1, e_2, \dots, e_K)$ 이라면, 0초부터 60초까지 매 시점의 플레이어7 상태에 이 임베딩을 추가 feature로 덧붙입니다. 이렇게 하면 해당 플레이어의 **캐릭터 고유 정보가 시간에 상관없이 항상 입력으로** 포함되고, CNN은 합성곱을 통

해 이 정보를 다른 동적 피쳐들과 함께 활용하게 됩니다. 실Implementation 측면에서는, **임베딩 행렬**을 정의하고 캐릭터 ID (정수)들을 임베딩lookup하여 shape (players, embed_dim)의 행렬을 얻은 뒤, 이를 시간축 길이만큼 복제하여 (players, embed_dim, T) 텐서로 만든 후 원래 status feature 텐서와 **채널 차원에서 결합 (concatenate)**합니다. Python의 Keras 등으로 표현하면 다음과 같은 흐름입니다:

```
# Pseudo-code for adding character ID embedding to input features
player_id_input = Input(shape=(10,), dtype='int32')           # 각 플레이어의 캐릭터 ID 10개
embed_layer = Embedding(input_dim=M, output_dim=K)           # M종 캐릭터를 K차원 임베딩
player_embeddings = embed_layer(player_id_input)               # 출력 shape: (batch, 10, K)
# 시간축 T=61만큼 임베딩을 반복하여 시계열 형태로 변환
player_emb_time = RepeatVector(T)(player_embeddings)           # shape: (batch, 10, T, K)
player_emb_time = Reshape((10, K, T))(player_emb_time)         # 임베딩을 players×K×T 텐서로
# 원본 status feature 입력 (batch, 10, F_status, T)와 concat
combined_input = Concatenate(axis=2)([status_input, player_emb_time]) # axis=feature 채널
```

위와 같은 절차로 **정적 특성(캐릭터 ID)**을 **동적 시계열 특징과 통합**할 수 있습니다. 이를 거친 combined_input 을 앞서 정한 CNN 아키텍처에 입력으로 사용하면, 모델은 합성곱 연산 내에서 **각 플레이어의 캐릭터 임베딩을 참고하여** 위치 변화를 예측할 수 있게 됩니다. (만약 (A) 방식으로 players를 채널로 완전히 펼쳤다면, 임베딩 벡터 역시 동일하게 펼쳐서 해당 player에 대응되는 채널 자리에 붙여주면 됩니다.)

임베딩 차원의 선택에 따라 표현력과 과적합 위험이 변하므로, 교차검증 등을 통해 적절한 임베딩 크기를 찾아야 합니다. 일반적으로 캐릭터 종류가 수십~수백이라면 4~16차원 정도 임베딩을 많이 사용합니다. **원-핫 vs 임베딩**에 대해, 캐릭터 종류가 극히 적다면 원-핫도 무방하지만, **임베딩이 학습 측면에서 더 유연하고** 병렬 플레이어 간 **캐릭터 유사성까지 학습**할 수 있으므로 권장됩니다.

데이터 전처리 전략: 불규칙 시간 및 결측치 처리

불규칙한 타임스탬프와 그로 인한 결측 데이터 처리는 본 과제의 중요한 난점입니다. 일반적인 CNN이나 RNN 모델은 **고정된 간격의 시계열**을 가정하고 설계되므로, 이벤트에 의해 중간에 비정기적 추가 기록이 생기면 이를 어떻게 다룰지 결정해야 합니다. 고려할 수 있는 전략들을 비교하면 다음과 같습니다:

- **선형 보간 (Linear Interpolation)으로 시간격자 보정:** 가장 단순한 접근은, **비정기 샘플을 정기화**하는 것입니다. 즉, 1초 단위의 기본 timeline을 유지하면서, 중간에 생긴 이벤트 시점의 값들을 **근처 시각에 반영하거나 보간**하는 것입니다. 가령 23.5초에 추가 데이터가 있다면, 23초와 24초 사이의 보간을 통해 해당 시점의 값을 끼워넣는 방식입니다. 이렇게 하면 모든 시계열 샘플이 동일한 길이와 간격(초당 1)으로 맞춰져, **결측치 없이 완전 격자형 데이터**가 얻어집니다. 이 방법의 장점은 **모델 단순화**로, CNN에 바로 입력하기 좋다는 것입니다. 그러나 **가정된 선형 변화**가 실제와 다를 수 있다는 문제가 있습니다. 연구에 따르면, 선부른 선형 보간은 데이터의 실제 동적 패턴(특히 이벤트에 의한 급변)을 왜곡하여 성능을 떨어뜨릴 수 있습니다¹⁷ ¹⁸. 보간이 적절하려면 **연속된 측정 사이의 변화가 선형에 가깝다**는 강한 가정이 필요한데, 게임 이벤트의 경우 그러지 않을 가능성이 높습니다. 따라서 보간을 하되, **이벤트로 인한 급변 시점에서는 보간 결과가 왜곡**임을 인지해야 합니다.

- **null/NaN 유지 + 마스킹:** 다른 접근은 **비정기 시점들을 추가로 포함**시켜 시간축을 확장하고, **존재하지 않는 값은 결측 표시**로 남기는 것입니다. 예를 들어 23.5초 이벤트를 **별도의 시간 index**로 추가하여 timeline을 세분화하고, 이 시점에 존재하지 않는 플레이어 상태는 NaN이나 특별값으로 채워두는 것입니다. 그런 다음 모델 측면에서 이 결측치를 다룰 수 있도록 **마스킹(mask)**나 **시간간격 정보**를 추가합니다. RNN 계열에서는 GRU-D와 같이 결측 마스크와 마지막 관측 후 경과시간 Δt 를 입력해 모델이 자체적으로 결측치를 처리하도록 한 연구가 있습니다¹⁹. CNN의 경우 명시적인 Δt 처리는 어렵지만, **결측 마스크를 추가 채널**로 제공하는 방법은 고려해볼 수 있습니다. 예를 들어 각 상태 값에 대해 결측이면 1, 관측된 값이면 0인 마스크를 동일한 shape의 채널로 입력하여, CNN이 해당 시점의 데이터 신뢰도를 학습하게 하는 것입니다. 하지만 일반 CNN은 NaN 값을 직접 다

루지 못하기 때문에, 실무적으로는 NaN을 0 등의 중립값으로 채운 뒤 마스크 채널로 결측을 표시하는 형태가 될 것입니다. 이 방식은 **이벤트에 의한 불규칙 패턴 자체를 학습**할 여지를 주지만, 구현 복잡성이 있고 데이터가 충분치 않으면 효과가 불확실합니다.

- **이벤트 인디케이터 추가:** 보간과 유사하게 정규화된 1초 간격을 쓰되, **이벤트 발생 여부를 별도 피쳐로 표시**하는 절충안도 있습니다. 즉 이벤트 시각을 가장 가까운 정규 시각으로 반올림하거나 해당 초에 플래그를 세워, "이 초에 특별한 이벤트가 있었음"을 나타내는 **binary feature**를 추가합니다. 모델은 이 피쳐를 통해 해당 시점에서의 변화가 일반적이지 않을 수 있음을 인지하게 됩니다. 예를 들어 23~24초 구간에 이벤트가 있었다면, 24초 시점의 이벤트 플래그=1 등의 방식입니다. 이 접근은 **추가 채널 한두 개만으로 이벤트 정보를 전달**하므로 간단하지만, 이벤트의 정밀한 시점을 잃고 근사화한다는 단점이 있습니다.

권장 전략: 위 방법들은 각기 trade-off가 있지만, **데이터 양과 이벤트 영향도**에 따라 조합해서 사용할 수도 있습니다. 예를 들어: - **기본적으로 선형 보간**으로 시계열을 균등화하되, - 이벤트로 인해 급격히 변한 피쳐의 경우 보간 대신 **이전 값으로 유지(forward-fill)**하거나 별도 처리, - 그리고 **이벤트 플래그 채널**을 추가하여 모델이 해당 구간을 특별히 다루게 하는 방안 등을 취할 수 있습니다.

실제 모델 개발 시에는 먼저 **간단한 전처리+모델로 baseline**을 만들어 보는 것이 좋습니다 ²⁰. 예를 들어 작은 보간으로 채운 데이터로 CNN을 훈련해보고, 성능을 측정한 뒤 부족하면 점진적으로 마스크나 고도화된 방법을 도입하는 것입니다. 또한, **타임스탬프 간격 자체를 하나의 특징으로 투입**하는 방법도 고려할 수 있습니다 (예: 각 시퀀스 스텝에 "이번 스텝 간 간격 Δt "를 피쳐로 포함). 이는 irregular 간격 정보를 직접 활용하는 아이디어로, TPCNN 등의 연구에서 제안된 방향이기도 합니다 ²¹ ²².

최신 연구 동향을 보면, **불규칙 시계열 전용 모델**들이 제안되고 있습니다. 예컨대 2023년 Kosma 등 ²³이 제안한 **Time-Parameterized CNN (TPCNN)**은 컨볼루션 커널에 **시간 정보를 직접 매개변수화**하여 irregular 데이터에서도 성능을 높였다고 보고합니다. 하지만 이러한 전용 기법은 구현이 복잡하므로, 현 단계에서는 **적절한 보간/보정으로 데이터를 정규 샘플링화**하여 표준 CNN을 적용하는 실용적인 접근을 권장합니다 ²¹. 다만, 보간으로 인한 정보 손실 가능성을 염두에 두고, **모델 검증 시 여러 전처리 버전을 비교**해 최적의 방법을 선택해야 합니다.

학습 방식 및 손실 함수 설정

마지막으로, 모델의 **학습 방법과 손실 함수 구성**에 관한 권장 사항입니다.

- **손실 함수 (Loss):** 예측 대상이 **연속값 좌표 (x,y)**이므로 **회귀 손실**을 사용합니다. 제시된 대로 **L2 노름 기반 손실**, 즉 **평균제곱오차(MSE)**가 기본 선택입니다. MSE는 예측 좌표와 실제 좌표 간의 $(\Delta x)^2 + (\Delta y)^2$ 오차를 최소화하도록 모델을 학습시킵니다. 만약 outlier 움직임 등으로 인해 MSE가 불안정하다면 **Huber Loss**와 같은 대안도 생각해볼 수 있지만, 일반적으로 위치 회귀에는 MSE가 잘 쓰입니다. 모델 출력이 $[x, y]$ 두 개이므로, 손실은 두 좌표의 MSE 합 또는 평균으로 계산됩니다. (L2 norm 자체는 $\sqrt{(\Delta x)^2 + (\Delta y)^2}$ 인데, 이는 미분이 까다로우므로 통상 $\text{MSE} = (\Delta x)^2 + (\Delta y)^2$ 를 최소화하면 동일한 최소점을 갖습니다.)
- **평가 지표:** 학습 중 모니터링을 위해서는 MSE의 루트인 **RMSE (Root MSE)**나 **평균 L2 거리**를 사용하면 직관적입니다 (예: 예측 좌표와 실제 좌표 간의 유클리드 거리). 예를 들어 validation 세트에서 평균 거리가 몇 (좌표단위)인지 확인하면 모델의 위치 오차를 쉽게 이해할 수 있습니다.
- **최적화 알고리즘:** Adam 옵티마이저 등 **적응형 경사하강법**이 효과적입니다. 초기 학습률은 0.001 정도로 시작하여 **Early Stopping**이나 **ReduceLROnPlateau** 스케줄러를 사용해 조절합니다 ²⁴. Epoch 수는 데이터 셋 크기에 따라 정해야 하나, validation loss를 모니터링하며 과적합 징후가 보이면 멈추는 것이 좋습니다

²⁵ .

- **데이터 분할 및 증강:** 게임 데이터 한 세트(한 경기)의 길이가 60초라면 비교적 짧습니다. 가능하면 여러 경기의 데이터를 모아 학습해야 일반화 능력이 생길 것입니다. 학습/검증/테스트 셋을 경기 단위로 분할하여, 특정 경기의 데이터를 본 모델이 그 경기 내 상황만 외워버리지 않도록 합니다. 증강의 경우, 좌표 데이터에 대해 작은 노이즈 추가나 좌표계 변환(예: 대칭 좌표계가 의미 있다면 좌우 대칭 변환) 등을 고려해볼 수 있습니다. 하지만 캐릭터 ID 등의 존재로 인해 마구 증강하기는 어렵고, 주어진 데이터 범위 내에서 학습하는 것이 무난합니다.

- **학습 과정 모니터링:** training loss와 validation loss의 추이를 관찰하여 과적합 여부를 판단합니다. 만약 validation loss 정체 혹은 상승하면서 training만 떨어지면 과적합이므로, 정규화 기법(Dropout 등) 강화나 학습률 조정이 필요합니다. 또한, 예측 결과를 리플레이 형태로 시각화해보면서, 7번 플레이어의 위치 예측이 시간이 진행됨에 따라 어떤 경향을 보이는지 인간적으로 이해해보는 것도 중요합니다.

- **추가 고려사항:**

- 모델이 특정 팀이나 특정 캐릭터 조합에 치우쳐 학습되지 않도록, 데이터의 다양성을 확보해야 합니다.
- 또한 플레이어 7번 자신의 과거 위치가 가장 큰 힌트일 수 있으므로, 모델이 그 자기 시계열에만 의존하지 않고 다른 플레이어와 상호작용도 고려하게 하는 것이 목표입니다. 이를 위해 Loss를 다각도로 분석해보는 것도 방법입니다. (예: 플레이어 7의 이동이 다른 플레이어와 충돌했을 때 오차가 큰지 등)

요약하면, MSE 손실과 Adam 최적화를 기본으로 설정하고, 체계적인 데이터 분할과 조기 종료(Early Stopping)로 일반화 성능을 확보하는 방향으로 학습을 진행합니다. 필요 시 하이퍼파라미터 탐색(learning rate, 필터 수, kernel_size 등)을 통해 최적 설정을 찾아낼 수도 있습니다.

결론 및 참고 자료

본 보고서에서는 게임 시뮬레이션 데이터로부터 특정 플레이어의 미래 위치를 예측하는 딥러닝 모델 설계를 위해, CNN 기반 접근법을 중점적으로 살펴보았습니다. 유사 연구 사례로부터 CNN이 시계열 특징 추출에 효과적이고, 다중 객체 상황에서도 활용되고 있음을 확인하였으며 ¹, 데이터 전처리 측면에서는 불규칙한 시간 간격 문제를 다루는 현실적인 방법(보간 및 마스킹 등)을 논의했습니다 ¹⁷. 특히 플레이어 ID와 같은 정적 정보를 임베딩하여 모델에 통합함으로써, 모델이 각 플레이어의 고유 특성을 학습하도록 하는 전략을 권장하였습니다 ¹⁶.

제안된 모델 아키텍처(1D-CNN + 임베딩 통합)는 비교적 간단하면서도, 주어진 과제의 핵심 요소들을 모두 반영하도록 구성되었습니다. 향후 구현 단계에서는 여기서 논의된 전처리와 모델 구조를 실제로 실험하면서, 성능을 정량적으로 평가하고 미세조정해야 할 것입니다. 필요한 경우 RNN(LSTM)이나 Graph CNN 등의 기법을 접목하여 플레이어 간 복잡한 상호작용을 더욱 잘 포착하도록 발전시킬 여지도 있습니다. 하지만 우선은 본 보고서의 내용을 토대로 기본 CNN 예측기의 성능을 확인하고, 이를 벤치마크로 삼아 개선을 도모하는 것을 추천드립니다.

참고 문헌 및 자료:

- Kosma et al., Time-Parameterized Convolutional Neural Networks for Irregularly Sampled Time Series, arXiv 2308.03210 (2023) - 불규칙 시계열에 CNN을 적용하는 최신 기법 소개 ²³ ²¹.
- Darjan Salaj, Deep Learning on Bad Time Series Data: Corrupt, Sparse, Irregular and Ugly, inovex Blog (2020) - 불규칙/결측 시계열 처리 총괄 정리 ¹⁷ ¹⁹.
- **HuggingFace TimeSeriesTransformer Docs** - 시계열 예측에서 정적 카테고리 특징을 임베딩으로 통합하는 방법 설명 ¹⁶.
- **StackExchange CrossValidated Q&A** - 다변량 시계열을 CNN에 적용하는 방법에 대한 논의 (1D vs 2D convolution에 대한 고찰) ⁹ ⁸.
- Cheong et al., Prediction of Defensive Player Trajectories in NFL Games with a CNN-LSTM Model, MIT Sloan Sports Conference (2021) - 1D-CNN으로 시계열 특징을 추출하고 LSTM으로 미래 경로를 예측한 사례 ¹.
- Fawaz et al., Time Series Classification from Scratch with Deep Neural Networks: A Strong Baseline, International Journal on Data Mining and Bioinformatics (2019) - 시계열 분류에 CNN을 사용하는 기법들을 정리 (시계열 CNN 구조 설계에 참고).

以上的 자료를 참고하면 보다 깊이 있는 이해와 구현에 도움이 될 것입니다. 모델 개발시 이론적인 고찰과 함께 **실험적 검증**을 병행하여 최적의 성능을 달성하길 바랍니다. 성공적인 예측 모델링을 기원합니다!

1 2 3 [assets.amazon.science](https://assets.amazon.science/8f/31/de231564410aa55e346d02c34c12/prediction-of-defensive-player-trajectories-in-nfl-games-with-defender-cnn-lstm-model.pdf)

<https://assets.amazon.science/8f/31/de231564410aa55e346d02c34c12/prediction-of-defensive-player-trajectories-in-nfl-games-with-defender-cnn-lstm-model.pdf>

4 5 6 [Temporal Convolutional Network — An Overview | by Amit Yadav | Medium](https://medium.com/@amit25173/temporal-convolutional-network-an-overview-4d2b6f03d6f8)

<https://medium.com/@amit25173/temporal-convolutional-network-an-overview-4d2b6f03d6f8>

7 [Win Rate Prediction in MOBA and FPS Games](https://drpress.org/ojs/index.php/HSET/article/view/18596/18136)

<https://drpress.org/ojs/index.php/HSET/article/view/18596/18136>

8 9 [neural networks - Why CNN is suitable for time-series data? - Cross Validated](https://stats.stackexchange.com/questions/550769/why-cnn-is-suitable-for-time-series-data)

<https://stats.stackexchange.com/questions/550769/why-cnn-is-suitable-for-time-series-data>

10 11 12 24 25 [Timeseries classification from scratch](https://keras.io/examples/timeseries/timeseries_classification_from_scratch/)

https://keras.io/examples/timeseries/timeseries_classification_from_scratch/

13 14 15 [How to Develop Convolutional Neural Network Models for Time Series Forecasting - MachineLearningMastery.com](https://www.machinelearningmastery.com/how-to-develop-convolutional-neural-network-models-for-time-series-forecasting/)

<https://www.machinelearningmastery.com/how-to-develop-convolutional-neural-network-models-for-time-series-forecasting/>

16 [Time Series Transformer](https://huggingface.co/docs/transformers/en/model_doc/time_series_transformer)

https://huggingface.co/docs/transformers/en/model_doc/time_series_transformer

17 18 19 20 [Deep Learning on Bad Time Series Data: Corrupt, Sparse, Irregular and Ugly - inovex GmbH](https://www.inovex.de/en/blog/deep-learning-time-series/)

<https://www.inovex.de/en/blog/deep-learning-time-series/>

21 22 23 [\[2308.03210\] Time-Parameterized Convolutional Neural Networks for Irregularly Sampled Time Series](https://arxiv.org/pdf/2308.03210)

<https://arxiv.org/pdf/2308.03210>